

APPENDIX A

DETAILED FORMULATION OF MULTI-MODEL PLACEMENT

This appendix provides the detailed formulation of multi-model placement under fixed workloads. Consider an idle heterogeneous device set $\mathcal{M}$ and a set of models $\mathcal{P}$ to be deployed. We assume that the representative workload of each model is known. Specifically, the workload of model p consists of b_p concurrent requests. For the j -th request ($1 \leq j \leq b_p$), s_j denotes the number of prefill tokens and r_j denotes the estimated number of generated output tokens.

To describe how models are distributed across devices, we define two model-device placement matrices: $\mathbf{W} = (w_{m,p}) \in \mathbb{Z}_{\geq 0}^{|\mathcal{M}| \times |\mathcal{P}|}$ and $\mathbf{N} = (n_{m,p}) \in \mathbb{Z}_{\geq 0}^{|\mathcal{M}| \times |\mathcal{P}|}$. Here, $w_{m,p}$ is the number of layers of model p assigned to device m , among which $n_{m,p}$ layers run on the GPU and the remaining $w_{m,p} - n_{m,p}$ layers run on the CPU. If $w_{m,p} = 0$, then $n_{m,p} = 0$ must also hold, indicating that device m does not participate in executing model p .

Let $\mathbf{X} = (x_{m,p}) \in \{0, 1\}^{|\mathcal{M}| \times |\mathcal{P}|}$ denote the head-device assignment matrix, where $x_{m,p} = 1$ iff device m is selected as the head device of model p . Each model is assigned exactly one unique head device. Additionally, we define three binary decision matrices: $\mathbf{A}_{\text{cpu}} = (a_{m,p}^{\text{cpu}}) \in \{0, 1\}^{|\mathcal{M}| \times |\mathcal{P}|}$, $\mathbf{A}_{\text{gpu}} = (a_{m,p}^{\text{gpu}}) \in \{0, 1\}^{|\mathcal{M}| \times |\mathcal{P}|}$, and $\mathbf{A} = (a_{m,p}) \in \{0, 1\}^{|\mathcal{M}| \times |\mathcal{P}|}$.

Here, $a_{m,p}^{\text{cpu}} = 1$ iff the CPU of device m executes at least one intermediate transformer layer of model p , and $a_{m,p}^{\text{gpu}} = 1$ iff the GPU of device m executes at least one intermediate transformer layer of model p . The input layer, output layer, and VLM visual sub-network are head-only modules and are modeled separately through the head-device variable $x_{m,p}$. The input and output layers are executed on the CPU of the head device.

The end-to-end latency for model p is $E_p = T_p^{\text{pre}} + T_p^{\text{dec}}$, where T_p^{pre} is the prefill latency and T_p^{dec} is the decode latency. The primary objective is to find a placement that minimizes the worst-case end-to-end latency across all models:

$$\min_{\mathbf{W}, \mathbf{N}, \mathbf{X}, \mathbf{A}_{\text{cpu}}, \mathbf{A}_{\text{gpu}}, \mathbf{A}, E^*} E^*, \quad \text{s.t. } E_p \leq E^*, \quad \forall p \in \mathcal{P}. \quad (4)$$

In implementation, we add a small tie-breaking term $\varepsilon \sum_{p \in \mathcal{P}} E_p$ so that, among placements with the same optimal E^* , the solver prefers the one with smaller aggregate latency. This tie-breaking term does not change the primary objective. We next derive T_p^{dec} and T_p^{pre} .

A. Decode Phase

For model p , all b_p requests decode in parallel, but request j stops after generating r_j tokens. The total number of decode steps is $R_{\max,p} = \max_j r_j$, determined by the longest request, while the number of active slots decreases as shorter requests finish. We therefore compute decode latency from the actual per-request workloads, rather than multiplying a fixed per-step cost by $R_{\max,p}$.

$$T_p^{\text{dec}} = \sum_{m \in \mathcal{M}} (T_{m,p}^{\text{comp,dec}} + T_{m,p}^{\text{mem,dec}} + T_{m,p}^{\text{comm,dec}}), \quad (5)$$

where $T_{m,p}^{\text{comp,dec}}$, $T_{m,p}^{\text{mem,dec}}$, and $T_{m,p}^{\text{comm,dec}}$ denote the computation, memory-access, and communication costs of model p on device m , accumulated over the $R_{\max,p}$ decode steps.

Estimation of computation latency $T_{m,p}^{\text{comp,dec}}$. For model p , device m executes $w_{m,p}$ transformer layers, where $n_{m,p}$ layers run on the GPU and $w_{m,p} - n_{m,p}$ layers run on the CPU. If m is the head device ($x_{m,p} = 1$), it also executes the output layer. We omit the input layer because embedding lookup has negligible cost. To estimate decode computation latency, we profile the per-layer FLOPs under each quantization format $q \in \mathcal{Q} = \{\text{Q4K}, \text{Q5K}, \text{Q6K}, \text{Q8_0}, \text{F16}, \text{F32}\}$. At decode step t of request j , the FLOPs of one transformer layer consist of two parts:

- Context-independent FLOPs. Operations such as QKV projection, output projection, and FFN incur a fixed per-token cost, denoted by $f_p^{q,\text{ci}}$.
- Context-dependent FLOPs. The attention cost depends on the current KV-cache length. At decode step t of request j , the cache contains $s_j + t$ entries, yielding $2n_{\text{head},p}(e_{k,p} + e_{v,p})(s_j + t)$ FLOPs in F16 and $7n_{\text{head},p}(s_j + t)$ FLOPs in F32.

Let $S_{\text{out},p} = \sum_j r_j$ be the total number of output tokens generated by model p . Then the total context-independent FLOPs of one layer are $S_{\text{out},p} f_p^{q,\text{ci}}$. For attention, summing over all requests and decode steps gives the total number of KV entries accessed: $T_{\text{kv},p} = \sum_{j=1}^{b_p} \sum_{t=1}^{r_j} (s_j + t) = \sum_{j=1}^{b_p} r_j \left(s_j + \frac{r_j+1}{2} \right)$. Therefore, the total per-layer attention FLOPs are $2n_{\text{head},p}(e_{k,p} + e_{v,p}) T_{\text{kv},p}$ in F16 and $7n_{\text{head},p} T_{\text{kv},p}$ in F32. Combining both parts, the total per-layer decode FLOPs under quantization format q are

$$F_p^{q,\text{dec}} = \begin{cases} S_{\text{out},p} f_p^{q,\text{ci}} + 2n_{\text{head},p}(e_{k,p} + e_{v,p}) T_{\text{kv},p}, & q = \text{F16}, \\ S_{\text{out},p} f_p^{q,\text{ci}} + 7n_{\text{head},p} T_{\text{kv},p}, & q = \text{F32}, \\ S_{\text{out},p} f_p^{q,\text{ci}}, & \text{otherwise.} \end{cases} \quad (6)$$

We further design a device profiler to measure the backend throughput of each device, denoted by $s_m^{\text{cpu},q}$ and $s_m^{\text{gpu},q}$ in FLOPS. Using the model and device profiles together, the total decode computation latency on device m is

$$T_{m,p}^{\text{comp,dec}} = (w_{m,p} - n_{m,p}) \sum_{q \in \mathcal{Q}} \frac{F_p^{q,\text{dec}}}{s_m^{\text{cpu},q}} + n_{m,p} \sum_{q \in \mathcal{Q}} \frac{F_p^{q,\text{dec}}}{s_m^{\text{gpu},q}} + x_{m,p} \sum_{q \in \mathcal{Q}} \frac{S_{\text{out},p} f_{p,\text{out}}^q}{s_m^{\text{cpu},q}}, \quad (7)$$

where $f_{p,\text{out}}^q$ is the per-token FLOPs of the output layer.

Estimation of memory-access latency $T_{m,p}^{\text{mem,dec}}$. This latency consists of three components: a) KV-cache copy time $T_{m,p}^{\text{kv_cpy,dec}}$, representing the time taken to copy the new token cache to the key-value cache storage within a specific device; b) device copy time $T_{m,p}^{\text{dev_cpy,dec}}$, representing the time taken to copy hidden states between the CPU and the GPU; c) device loading time $T_{m,p}^{\text{dev_load,dec}}$, representing the time taken to load data from RAM or VRAM into the processing cores of the CPU or GPU.

a) *KV-cache copy time.* At each decoding step, every executed layer appends one new key and one new value to the KV-cache. For model p , the key and value sizes are $h_{k,p}e_{k,p}$ and $h_{v,p}e_{v,p}$, respectively, where $h_{k,p}$ and $h_{v,p}$ denote the numbers of key and value heads, and $e_{k,p}$ and $e_{v,p}$ denote the corresponding per-head dimensions. Thus, each layer writes $(h_{k,p}e_{k,p} + h_{v,p}e_{v,p})$ values per output token. Assuming F16 storage, this corresponds to $\kappa_p = 2(h_{k,p}e_{k,p} + h_{v,p}e_{v,p})$ bytes per token per layer.

In the device profiler, we measure the time to copy κ_p bytes within CPU memory and GPU memory, denoted by $t_{m,p}^{\text{kv_cpy,cpu}}$ and $t_{m,p}^{\text{kv_cpy,gpu}}$, respectively. Therefore, generating $S_{\text{out},p}$ output tokens incurs $T_{m,p}^{\text{kv_cpy,dec}} = S_{\text{out},p} \left[(w_{m,p} - n_{m,p}) t_{m,p}^{\text{kv_cpy,cpu}} + n_{m,p} t_{m,p}^{\text{kv_cpy,gpu}} \right]$.

b) *device copy time.* This latency arises when the GPU is enabled, involving copying the input from RAM to VRAM and the output from VRAM back to RAM. For model p , both the input and output hidden states have dimension $(e_p, 1)$ and are typically stored in F32, i.e., $4e_p$ bytes. The device profiler measures the latency of copying $4e_p$ bytes from RAM to VRAM and from VRAM to RAM, denoted by $t_{m,p}^{\text{ram-vram}}$ and $t_{m,p}^{\text{vram-ram}}$, respectively. Hence, $T_{m,p}^{\text{dev_cpy,dec}} = a_{m,p}^{\text{gpu}} S_{\text{out},p} (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}})$.

c) *device loading time.* Processing cores must first load data from main memory (RAM or VRAM) into registers before executing instructions. This process incurs latency. However, theoretical memory bandwidth is insufficient for estimating this latency, because actual memory throughput is shaped by access patterns, cache hierarchy, and runtime overheads. To capture these effects, our device profiler implements a custom operator to read data from RAM/VRAM into the processors' registers. By measuring the latency of this operator with data volumes similar to the tensor sizes used in transformer-based models, we estimate the practical memory throughput, denoted as $\{\mathcal{T}_m^{\text{cpu}}, \mathcal{T}_m^{\text{gpu}}\}$, across heterogeneous devices.

Next, we estimate the volume of data that must be loaded into registers during each decoding step. This data volume primarily consists of two parts: the model weight data and the KV-cache. The model weights are loaded once per decoding step. In our model profiler, we record the total size of the input layer weights as ψ_{ip} , the output layer weights as ψ_{op} , and the weights for each intermediate layer as ψ_p . KV-cache reads depend on the context depth. At decode step t of request j , attention reads $s_j + t$ KV entries; summing over all requests and steps yields $T_{\text{kv},p}$ total entries, corresponding to $\kappa_p T_{\text{kv},p}$ bytes per layer. Then, $T_{m,p}^{\text{dev_load,dec}} = \left(\frac{w_{m,p} - n_{m,p}}{\mathcal{T}_m^{\text{cpu}}} + \frac{n_{m,p}}{\mathcal{T}_m^{\text{gpu}}} \right) [R_{\text{max},p} \cdot \psi_p + \kappa_p \cdot T_{\text{kv},p}] + x_{m,p} \frac{S_{\text{out},p} \cdot \psi_{ip} / V_p + R_{\text{max},p} \cdot \psi_{op}}{\mathcal{T}_m^{\text{cpu}}}$. The first head-device term accounts for embedding table lookups ($S_{\text{out},p}$ tokens, each loading ψ_{ip}/V_p bytes), and the second accounts for output-layer weight loading over $R_{\text{max},p}$ steps.

Now we can combine the three latency components and give the formal definition of the memory access latency $T_{m,p}^{\text{mem,dec}}$:

$$T_{m,p}^{\text{mem,dec}} = S_{\text{out},p} \left[(w_{m,p} - n_{m,p}) t_{m,p}^{\text{kv_cpy,cpu}} + n_{m,p} t_{m,p}^{\text{kv_cpy,gpu}} \right] + a_{m,p}^{\text{gpu}} S_{\text{out},p} (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}}) + \left(\frac{w_{m,p} - n_{m,p}}{\mathcal{T}_m^{\text{cpu}}} + \frac{n_{m,p}}{\mathcal{T}_m^{\text{gpu}}} \right) [R_{\text{max},p} \psi_p + \kappa_p T_{\text{kv},p}] + x_{m,p} \frac{S_{\text{out},p} \psi_{ip} / V_p + R_{\text{max},p} \psi_{op}}{\mathcal{T}_m^{\text{cpu}}}. \quad (8)$$

Estimation of network communication latency $T_{m,p}^{\text{comm,dec}}$. In MaestroServe, devices are logically interconnected in a ring structure, where each device receives input from its predecessor, completes the computation, and sends the result to its successor. For each request, the result consists of e_p values in F32 format, totaling $4e_p$ bytes. Therefore, by measuring the latency $t_{m,p}^{\text{comm}}$ for transmitting $4e_p$ bytes between adjacent devices, we can estimate the network communication latency on device m as:

$$T_{m,p}^{\text{comm,dec}} = a_{m,p} S_{\text{out},p} \cdot t_{m,p}^{\text{comm}}. \quad (9)$$

By aggregating these latencies, we obtain the total decode latency for model p :

$$T_p^{\text{dec}} = \sum_{m \in \mathcal{M}} \left[x_{m,p} \left(\sum_{q \in \mathcal{Q}} \frac{S_{\text{out},p} f_{p,\text{out}}^q}{s_m^{\text{cpu},q}} + \frac{S_{\text{out},p} \psi_{i,p}/V_p + R_{\text{max},p} \psi_{o,p}}{\mathcal{T}_m^{\text{cpu}}} \right) + (w_{m,p} - n_{m,p}) \left(\sum_{q \in \mathcal{Q}} \frac{F_p^{q,\text{dec}}}{s_m^{\text{cpu},q}} + S_{\text{out},p} t_{m,p}^{\text{kv_cpy,cpu}} \right) \right. \\ \left. + \frac{R_{\text{max},p} \psi_p + \kappa_p T_{\text{kv},p}}{\mathcal{T}_m^{\text{cpu}}} \right) + n_{m,p} \left(\sum_{q \in \mathcal{Q}} \frac{F_p^{q,\text{dec}}}{s_m^{\text{gpu},q}} + S_{\text{out},p} t_{m,p}^{\text{kv_cpy,gpu}} + \frac{R_{\text{max},p} \psi_p + \kappa_p T_{\text{kv},p}}{\mathcal{T}_m^{\text{gpu}}} \right) \\ \left. + a_{m,p}^{\text{gpu}} S_{\text{out},p} (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}}) + a_{m,p} S_{\text{out},p} t_{m,p}^{\text{comm}} \right]. \quad (10)$$

B. Prefill phase

For model p , each request j has a prefill length $s_j = s_j^{\text{txt}} + s_j^{\text{vis}}$, where s_j^{txt} is the number of text tokens and s_j^{vis} the number of visual tokens (nonzero only for VLMs). We define $S_{\text{pre},p}^{\text{txt}} = \sum_{j=1}^{b_p} s_j^{\text{txt}}$, $S_{\text{pre},p}^{\text{vis}} = \sum_{j=1}^{b_p} s_j^{\text{vis}}$, $S_{\text{pre},p} = S_{\text{pre},p}^{\text{txt}} + S_{\text{pre},p}^{\text{vis}}$. Prefill is executed in continuous-batching rounds, each processing at most n_{ubatch} tokens. Therefore, model p requires $N_{\text{pre},p} = \left\lceil \frac{S_{\text{pre},p}}{n_{\text{ubatch}}} \right\rceil$ rounds in total. The overall prefill latency is then

$$T_p^{\text{pre}} = \sum_{m \in \mathcal{M}} \left(T_{m,p}^{\text{comp,pre}} + T_{m,p}^{\text{mem,pre}} + T_{m,p}^{\text{comm,pre}} \right). \quad (11)$$

Estimation of prefill computation latency $T_{m,p}^{\text{comp,pre}}$. The prefill computation consists of two sequential stages: (i) the visual sub-network (VLM only), which converts raw image patches into visual tokens on the head device's GPU; and (ii) the LLM backbone, which processes all $S_{\text{pre},p}$ tokens across all assigned devices.

a) Stage 1: Visual sub-network (VLM only): For model p with $I_p^{\text{VL}} = 1$, the head device's GPU runs the ViT encoder (L_p^{ve} layers), spatial merger, and linear projector. The model profiler records their respective FLOPs as f_p^{ve} (per encoder layer), f_p^{merge} , and f_p^{proj} . Since the mmproj weights use a single precision format q_{vit} (typically F16), all FLOPs are evaluated at the GPU throughput $s_m^{\text{gpu},q_{\text{vit}}}$:

$$T_{m,p}^{\text{vis}} = x_{m,p} \cdot \mathbb{I}_p^{\text{VL}} \cdot \frac{L_p^{\text{ve}} f_p^{\text{ve}} + f_p^{\text{merge}} + f_p^{\text{proj}}}{s_m^{\text{gpu},q_{\text{vit}}}}. \quad (12)$$

b) Stage 2: LLM backbone: The model profiler counts the per-layer prefill FLOPs $\{f_p^{q,\text{pre}}\}_{q \in \mathcal{Q}}$ for model p over its b_p requests. In llama.cpp, prefill attention is performed independently for each sequence: request j attends only to its own s_j prompt tokens. Therefore, the attention cost grows quadratically with each request length, and $f_p^{q,\text{pre}}$ includes a $\sum_{j=1}^{b_p} s_j^2$ term rather than $S_{\text{pre},p}^2$. The output-layer prefill FLOPs equal $b_p \cdot f_{p,\text{out}}^q$, since only the last token of each request requires logits. Thus, the prefill computation latency on device m is

$$T_{m,p}^{\text{comp,pre}} = (w_{m,p} - n_{m,p}) \sum_{q \in \mathcal{Q}} \frac{f_p^{q,\text{pre}}}{s_m^{\text{cpu},q}} + n_{m,p} \sum_{q \in \mathcal{Q}} \frac{f_p^{q,\text{pre}}}{s_m^{\text{gpu},q}} + x_{m,p} \sum_{q \in \mathcal{Q}} \frac{b_p \cdot f_{p,\text{out}}^q}{s_m^{\text{cpu},q}} + T_{m,p}^{\text{vis}}. \quad (13)$$

Estimation of prefill memory-access latency $T_{m,p}^{\text{mem,pre}}$. The prefill memory-access latency follows the same three components as the decode memory-access model: KV-cache copy, device copy, and device loading. The token count is $S_{\text{pre},p}$. For KV-cache copy time, all $S_{\text{pre},p}$ tokens copy their KV entries simultaneously: $T_{m,p}^{\text{kv_cpy,pre}} = S_{\text{pre},p} \left[(w_{m,p} - n_{m,p}) t_{m,p}^{\text{kv_cpy,cpu}} + n_{m,p} t_{m,p}^{\text{kv_cpy,gpu}} \right]$. For device copy time, the hidden-state tensor has shape $(e_p, S_{\text{pre},p})$: $T_{m,p}^{\text{dev_cpy,pre}} = a_{m,p}^{\text{gpu}} \cdot S_{\text{pre},p} (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}})$. For device loading time, each of the $N_{\text{pre},p}$ prefill rounds loads model weights once. The per-layer KV cache read during causal prefill attention totals $\kappa_p \cdot S_{\text{pre},p}^{\text{quad}}$ bytes, where $S_{\text{pre},p}^{\text{quad}} = \sum_{j=1}^{b_p} \frac{s_j(s_j-1)}{2}$, counts the total prefill KV reads (token t in request j attends to $t-1$ previous tokens). For VLMs, the visual sub-network weights ψ_p^{ve} are additionally loaded into GPU registers: $T_{m,p}^{\text{dev_load,pre}} = \left(\frac{w_{m,p} - n_{m,p}}{\mathcal{T}_m^{\text{cpu}}} + \frac{n_{m,p}}{\mathcal{T}_m^{\text{gpu}}} \right) [N_{\text{pre},p} \cdot \psi_p + \kappa_p \cdot S_{\text{pre},p}^{\text{quad}}] + x_{m,p} \frac{S_{\text{pre},p}^{\text{txt}} \cdot \psi_{i,p}/V_p + N_{\text{pre},p} \cdot \psi_{o,p}}{\mathcal{T}_m^{\text{out}}} + x_{m,p} \cdot \mathbb{I}_p^{\text{VL}} \cdot \frac{\psi_p^{\text{ve}}}{\mathcal{T}_m^{\text{gpu}}}$.

Therefore, the prefill memory-access latency is the sum of the above three components:

$$T_{m,p}^{\text{mem,pre}} = S_{\text{pre},p} \left[(w_{m,p} - n_{m,p}) t_{m,p}^{\text{kv_cpy,cpu}} + n_{m,p} t_{m,p}^{\text{kv_cpy,gpu}} \right] + a_{m,p}^{\text{gpu}} \cdot S_{\text{pre},p} (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}}) \\ + \left(\frac{w_{m,p} - n_{m,p}}{\mathcal{T}_m^{\text{cpu}}} + \frac{n_{m,p}}{\mathcal{T}_m^{\text{gpu}}} \right) [N_{\text{pre},p} \cdot \psi_p + \kappa_p \cdot S_{\text{pre},p}^{\text{quad}}] + x_{m,p} \frac{S_{\text{pre},p}^{\text{txt}} \cdot \psi_{i,p}/V_p + N_{\text{pre},p} \cdot \psi_{o,p}}{\mathcal{T}_m^{\text{out}}} + x_{m,p} \cdot \mathbb{I}_p^{\text{VL}} \cdot \frac{\psi_p^{\text{ve}}}{\mathcal{T}_m^{\text{gpu}}}. \quad (14)$$

Estimation of prefill communication latency $T_{m,p}^{\text{comm,pre}}$. All $S_{\text{pre},p}$ tokens are forwarded simultaneously:

$$T_{m,p}^{\text{comm,pre}} = a_{m,p} S_{\text{pre},p} \cdot t_{m,p}^{\text{comm}}. \quad (15)$$

By aggregating these latencies, we obtain the total prefill latency for model p :

$$\begin{aligned} T_p^{\text{pre}} = \sum_{m \in \mathcal{M}} \left[x_{m,p} \left(\sum_{q \in \mathcal{Q}} \frac{b_p \cdot f_{p,\text{out}}^q}{s_m^{\text{cpu},q}} + \frac{S_{\text{pre},p}^{\text{txt}} \psi_{i,p}/V_p + N_{\text{pre},p} \psi_{o,p}}{\mathcal{T}_m^{\text{cpu}}} + \mathbb{I}_p^{\text{VL}} \left(\frac{L_p^{\text{ve}} f_p^{\text{ve}} + f_p^{\text{merge}} + f_p^{\text{proj}}}{s_m^{\text{gpu},q_{\text{vit}}}} + \frac{\psi_p^{\text{ve}}}{\mathcal{T}_m^{\text{gpu}}} \right) \right) + (w_{m,p} - n_{m,p}) \right. \\ \left. \left(\sum_{q \in \mathcal{Q}} \frac{f_p^{q,\text{pre}}}{s_m^{\text{cpu},q}} + S_{\text{pre},p} t_{m,p}^{\text{kv_cpy,cpu}} + \frac{N_{\text{pre},p} \psi_p + \kappa_p S_{\text{pre},p}^{\text{quad}}}{\mathcal{T}_m^{\text{cpu}}} \right) + n_{m,p} \left(\sum_{q \in \mathcal{Q}} \frac{f_p^{q,\text{pre}}}{s_m^{\text{gpu},q}} + S_{\text{pre},p} t_{m,p}^{\text{kv_cpy,gpu}} + \frac{N_{\text{pre},p} \psi_p + \kappa_p S_{\text{pre},p}^{\text{quad}}}{\mathcal{T}_m^{\text{gpu}}} \right) \right. \\ \left. + a_{m,p}^{\text{gpu}} S_{\text{pre},p} (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}}) + a_{m,p} (S_{\text{pre},p} t_{m,p}^{\text{comm}}) \right]. \quad (16) \end{aligned}$$

C. Unified End-to-End Coefficients

Substituting $E_p = T_p^{\text{pre}} + T_p^{\text{dec}}$, we obtain

$$\begin{aligned} E_p = \sum_{m \in \mathcal{M}} \left[x_{m,p} \left(\sum_{q \in \mathcal{Q}} \frac{(b_p + S_{\text{out},p}) f_{p,\text{out}}^q}{s_m^{\text{cpu},q}} + \frac{(S_{\text{pre},p}^{\text{txt}} + S_{\text{out},p}) \psi_{i,p}/V_p + N_{\text{steps},p} \psi_{o,p}}{\mathcal{T}_m^{\text{cpu}}} + \mathbb{I}_p^{\text{VL}} \left(\frac{L_p^{\text{ve}} f_p^{\text{ve}} + f_p^{\text{merge}} + f_p^{\text{proj}}}{s_m^{\text{gpu},q_{\text{vit}}}} + \frac{\psi_p^{\text{ve}}}{\mathcal{T}_m^{\text{gpu}}} \right) \right) \right. \\ \left. + (w_{m,p} - n_{m,p}) \left(\sum_{q \in \mathcal{Q}} \frac{f_p^{q,\text{pre}} + F_p^{q,\text{dec}}}{s_m^{\text{cpu},q}} + (S_{\text{pre},p} + S_{\text{out},p}) t_{m,p}^{\text{kv_cpy,cpu}} + \frac{N_{\text{steps},p} \psi_p + \kappa_p (S_{\text{pre},p}^{\text{quad}} + T_{\text{kv},p})}{\mathcal{T}_m^{\text{cpu}}} \right) \right. \\ \left. + n_{m,p} \left(\sum_{q \in \mathcal{Q}} \frac{f_p^{q,\text{pre}} + F_p^{q,\text{dec}}}{s_m^{\text{gpu},q}} + (S_{\text{pre},p} + S_{\text{out},p}) t_{m,p}^{\text{kv_cpy,gpu}} + \frac{N_{\text{steps},p} \psi_p + \kappa_p (S_{\text{pre},p}^{\text{quad}} + T_{\text{kv},p})}{\mathcal{T}_m^{\text{gpu}}} \right) \right. \\ \left. + a_{m,p}^{\text{gpu}} (S_{\text{pre},p} + S_{\text{out},p}) (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}}) + a_{m,p} ((S_{\text{pre},p} + S_{\text{out},p}) t_{m,p}^{\text{comm}}) \right], \quad (17) \end{aligned}$$

where $N_{\text{steps},p} = N_{\text{pre},p} + R_{\text{max},p}$ denotes the total number of weight-loading steps across both phases.

The above expression is linear in the five groups of decision variables $w_{m,p} - n_{m,p}$, $n_{m,p}$, $a_{m,p}^{\text{gpu}}$, $a_{m,p}$, and $x_{m,p}$. We read off their coefficients to write E_p in the compact form

$$E_p = \sum_{m \in \mathcal{M}} [\alpha_{m,p} w_{m,p} + \beta_{m,p} n_{m,p} + \gamma_{m,p} a_{m,p}^{\text{gpu}} + \xi_{m,p} a_{m,p} + \delta_{m,p} x_{m,p}]. \quad (18)$$

The coefficient $\alpha_{m,p}$ captures the marginal cost of assigning one additional layer to the CPU of device m , aggregating the computation, KV-cache copy, and device loading latencies across both prefill and decode phases:

$$\alpha_{m,p} = \underbrace{\sum_{q \in \mathcal{Q}} \frac{f_p^{q,\text{pre}} + F_p^{q,\text{dec}}}{s_m^{\text{cpu},q}}}_{\text{computation latency}} + \underbrace{(S_{\text{pre},p} + S_{\text{out},p}) t_{m,p}^{\text{kv_cpy,cpu}}}_{\text{KV-cache copy latency}} + \underbrace{\frac{N_{\text{steps},p} \psi_p + \kappa_p (S_{\text{pre},p}^{\text{quad}} + T_{\text{kv},p})}{\mathcal{T}_m^{\text{cpu}}}}_{\text{device loading latency}}. \quad (19)$$

The device loading term consists of three parts: $N_{\text{steps},p} \psi_p$ for model weights loaded once per step across all prefill rounds and decode steps, $\kappa_p S_{\text{pre},p}^{\text{quad}}$ for KV cache reads during causal prefill, and $\kappa_p T_{\text{kv},p}$ for KV cache reads during decode.

The coefficient $\beta_{m,p}$ captures the marginal latency difference of moving one layer from CPU to GPU. Its three terms mirror those of $\alpha_{m,p}$ —computation, KV-cache copy, and device loading—each expressed as a GPU-minus-CPU differential:

$$\begin{aligned} \beta_{m,p} = \underbrace{\sum_{q \in \mathcal{Q}} (f_p^{q,\text{pre}} + F_p^{q,\text{dec}}) \left(\frac{1}{s_m^{\text{gpu},q}} - \frac{1}{s_m^{\text{cpu},q}} \right)}_{\Delta \text{ computation latency}} + \underbrace{(S_{\text{pre},p} + S_{\text{out},p}) (t_{m,p}^{\text{kv_cpy,gpu}} - t_{m,p}^{\text{kv_cpy,cpu}})}_{\Delta \text{ KV-cache copy latency}} \\ + \underbrace{[N_{\text{steps},p} \psi_p + \kappa_p (S_{\text{pre},p}^{\text{quad}} + T_{\text{kv},p})] \left(\frac{1}{\mathcal{T}_m^{\text{gpu}}} - \frac{1}{\mathcal{T}_m^{\text{cpu}}} \right)}_{\Delta \text{ device loading latency}}. \quad (20) \end{aligned}$$

Typically $\beta_{m,p} < 0$ since GPUs are faster than CPUs, so assigning more layers to the GPU reduces latency.

The coefficient $\gamma_{m,p}$ corresponds to the device copy latency (RAM $\leftrightarrow$ VRAM) defined above. It is incurred once per token whenever a discrete GPU is used:

$$\gamma_{m,p} = (S_{\text{pre},p} + S_{\text{out},p}) (t_{m,p}^{\text{ram-vram}} + t_{m,p}^{\text{vram-ram}}). \quad (21)$$

The coefficient $\xi_{m,p}$ corresponds to the communication latency. It is incurred on every device participating in model p :

$$\xi_{m,p} = (S_{\text{pre},p} + S_{\text{out},p}) t_{m,p}^{\text{comm}}. \quad (22)$$

Finally, $\delta_{m,p}$ captures the additional latency borne exclusively by the head device. It consists of the output-layer computation latency, the input embedding and output weight device loading latency. For VLMs, it also consists of the visual sub-network computation and weight loading latency:

$$\delta_{m,p} = \underbrace{\sum_{q \in \mathcal{Q}} \frac{(b_p + S_{\text{out},p}) f_{p,\text{out}}^q}{s_m^{\text{cpu},q}}}_{\text{output computation latency}} + \underbrace{\frac{(S_{\text{pre},p}^{\text{txt}} + S_{\text{out},p}) \psi_{i,p}/V_p + N_{\text{steps},p} \psi_{o,p}}{\mathcal{T}_m^{\text{cpu}}}}_{\text{I/O layer device loading latency}} + \underbrace{\mathbb{I}_p^{\text{VL}} \left(\frac{L_p^{\text{ve}} f_p^{\text{ve}} + f_p^{\text{merge}} + f_p^{\text{proj}}}{s_m^{\text{gpu},q_{\text{vit}}}} + \frac{\psi_p^{\text{ve}}}{\mathcal{T}_m^{\text{gpu}}} \right)}_{\text{visual sub-network latency (VLM only)}}. \quad (23)$$

The output computation sums b_p tokens from prefill (last token per slot) and $S_{\text{out},p}$ tokens from decode. The I/O layer loading term uses $S_{\text{pre},p}^{\text{txt}}$ for input embedding (text prefill tokens only, since visual tokens come from the ViT encoder) plus $S_{\text{out},p}$ decode tokens, and loads output weights once per step ($N_{\text{steps},p}$ times).

D. Sufficient-Memory Constraints

Tangram has analyzed disk loading latency caused by mmap page reclamation under heterogeneous operating systems. In MaestroServe, we focus on relatively smaller models and assume that disk loading will not be triggered during the decoding process. To ensure the validity of this assumption, we must explicitly impose OS-specific memory sufficiency constraints on the placement variables.

Let $\psi'_p = \psi_p + 2(h_{k,p}e_{k,p} + h_{v,p}e_{v,p})T_{\text{kv},p}$, which denotes the per-layer max resident memory, including both model weights and the KV-cache. Let $\psi'_{p,\text{io}} = \frac{b_p \psi_{i,p}}{V} + \psi_{o,p}$. Therefore, the CPU-side sufficient-memory condition is $\sum_{p \in \mathcal{P}} [(w_{m,p} - n_{m,p})\psi'_p + x_{m,p}\psi'_{p,\text{io}} + c_p^{\text{cpu}} a_{m,p}^{\text{cpu}}] \leq d_m^{\text{ram}}, \forall m$, and the GPU-side capacity must also satisfy $\sum_{p \in \mathcal{P}} [n_{m,p}\psi'_p + c_p^{\text{gpu}} a_{m,p}^{\text{gpu}} + x_{m,p}\psi_p^{\text{ve}} \cdot \mathbb{I}_p^{\text{VL}}] \leq d_m^{\text{vram}}, \forall m$.

E. Complete MILP Formulation

Now, we put it all together to present a formal definition of the optimization problem:

$$\min_{\mathbf{W}, \mathbf{N}, \mathbf{X}, \mathbf{A}_{\text{cpu}}, \mathbf{A}_{\text{gpu}}, \mathbf{A}, E^*} E^* + \epsilon \sum_{p \in \mathcal{P}} E_p \quad (24)$$

$$\text{s.t. } E_p \leq E^*, \quad \forall p, \quad (25)$$

$$E_p = \sum_{m \in \mathcal{M}} [\alpha_{m,p} w_{m,p} + \beta_{m,p} n_{m,p} + \gamma_{m,p} a_{m,p}^{\text{gpu}} + \xi_{m,p} a_{m,p} + \delta_{m,p} x_{m,p}], \quad \forall p, \quad (26)$$

$$0 \leq n_{m,p} \leq w_{m,p} \leq L_p, \quad \sum_{m \in \mathcal{M}} w_{m,p} = L_p, \quad \forall m, p, \quad (27)$$

$$\sum_{p \in \mathcal{P}} a_{m,p}^{\text{cpu}} \leq 1, \quad \sum_{p \in \mathcal{P}} a_{m,p}^{\text{gpu}} \leq 1, \quad \forall m, \quad (28)$$

$$a_{m,p}^{\text{cpu}} \leq w_{m,p} - n_{m,p} \leq L_p a_{m,p}^{\text{cpu}}, \quad \forall m, p, \quad (29)$$

$$a_{m,p}^{\text{gpu}} \leq n_{m,p} \leq L_p a_{m,p}^{\text{gpu}}, \quad \forall m, p, \quad (30)$$

$$a_{m,p}^{\text{cpu}} \leq a_{m,p}, \quad a_{m,p}^{\text{gpu}} \leq a_{m,p} \leq a_{m,p}^{\text{cpu}} + a_{m,p}^{\text{gpu}}, \quad \forall m, p, \quad (31)$$

$$\sum_{m \in \mathcal{M}} x_{m,p} = 1, \quad \sum_{p \in \mathcal{P}} x_{m,p} \leq 1, \quad x_{m,p} \leq a_{m,p}, \quad \forall m, p, \quad (32)$$

$$x_{m,p} \mathbb{I}_p^{\text{VL}} \leq \mathbb{I}_m^{\text{cuda}}, \quad \forall m, p, \quad (33)$$

$$a_{m,p}^{\text{gpu}} \geq x_{m,p} \cdot \mathbb{I}_p^{\text{VL}}, \quad w_{m,p} \geq D_p \cdot x_{m,p} \cdot \mathbb{I}_p^{\text{VL}}, \quad \forall m, p, \quad (34)$$

$$\psi'_p = \psi_p + 2(h_{k,p}e_{k,p} + h_{v,p}e_{v,p})T_{\text{kv},p}, \quad \psi'_{p,\text{io}} = b_p \psi_{i,p}/V_p + \psi_{o,p}, \quad \forall p, \quad (35)$$

$$\sum_{p \in \mathcal{P}} [(w_{m,p} - n_{m,p})\psi'_p + x_{m,p}\psi'_{p,\text{io}} + c_p^{\text{cpu}} a_{m,p}^{\text{cpu}}] \leq d_m^{\text{ram}}, \quad \forall m, \quad (36)$$

$$\sum_{p \in \mathcal{P}} [n_{m,p}\psi'_p + c_p^{\text{gpu}} a_{m,p}^{\text{gpu}} + x_{m,p}\psi_p^{\text{ve}} \mathbb{I}_p^{\text{VL}}] \leq d_m^{\text{vram}}, \quad \forall m, \quad (37)$$

$$w_{m,p}, n_{m,p} \in \mathbb{Z}_{\geq 0}, \quad \forall m, p, \quad (38)$$

$$x_{m,p}, a_{m,p}, a_{m,p}^{\text{cpu}}, a_{m,p}^{\text{gpu}} \in \{0, 1\}, \quad \forall m, p. \quad (39)$$

Constraint (25) defines E^* as an upper bound on the latency of all models. Constraint (27) ensures that the number of GPU layers on each device does not exceed the assigned layers, and that all L_p layers of model p are placed exactly once. Constraint (28) allows each CPU or GPU backend to serve at most one model. Constraints (29) and (30) link backend indicators with layer allocations. They make $a_{m,p}^{\text{cpu}}$ and $a_{m,p}^{\text{gpu}}$ active iff device m executes at least one CPU or GPU layer of model p , respectively. Constraint (31) defines $a_{m,p}$ as the union of CPU and GPU participation. Constraint (32) selects one participating head device for each model and ensures that each device serves as the head for at most one model. For VLMs, Constraint (33) requires the selected head device to support CUDA. Constraint (34) further ensures that the VLM head uses a GPU backend and places at least D_p layers on the head device, as required by the visual-processing pipeline. Constraints (36) and (37) enforce RAM and VRAM feasibility on each device. The RAM constraint accounts for CPU-side transformer layers, head-device I/O memory, and CPU runtime buffers. The VRAM constraint accounts for GPU-side transformer layers, GPU runtime buffers, and the visual encoder memory required by VLMs. Finally, Constraints (38)–(39) specify the domains of integer layer-allocation variables and binary assignment variables.

Since the objective and all constraints are linear in the decision variables, the formulation is a mixed-integer linear program and can be solved by an optimizer such as HiGHS.